

PROGRAMMING 2B

PoE PART 1

SYSTEM PLANNING AND DATABASE

DETAILED ASSESSMENT BREAKDOWN AND STEP-BY-STEP STUDENT GUIDE

Total: 100 Marks

**Important: Part 1 is a planning and database submission.
Do not build the API or MVC application yet.**

Prepared as a student-facing guide based on the official PROG6212 PoE Part 1 brief and rubric.

1. PART 1 OVERVIEW

The purpose of Part 1 is to plan the RaceDay system properly before development starts. RaceDay is a web-based event management system for South African road running, walking and cycling events.

For Part 1, I am not expecting you to develop the API or the MVC application yet. I am expecting you to demonstrate that you understand the system well enough to design the data model, plan the future RESTful API, create the SQL database script, manage your work correctly on GitHub, and explain your decisions in a video presentation.

Your main Part 1 deliverables are:

- Entity Relationship Diagram (ERD).
- API Endpoint Plan.
- SQL Database Script.
- GitHub repository with meaningful commit history.
- GitHub Actions CI/CD validation.
- README file.
- Unlisted YouTube video presentation.

The most important idea to understand is that Part 1 becomes the blueprint for Part 2. The ERD, endpoint plan and SQL script should therefore agree with one another. If your planning is inconsistent now, you will create unnecessary problems when you start building the API.

2. UNDERSTAND RACEDAY BEFORE YOU START

Before drawing an ERD or writing SQL, read the entire PoE, including Parts 2 and 3. I recommend this because the database and API plan you design now must support the functionality you will develop later.

RaceDay supports two user roles:

Organiser

- Create, edit and delete events.
- Manage event categories.
- View event enrolments.
- Capture participant results.
- View information relating to events they manage.

Participant

- Create an account and log in.
- Browse available events.
- Enter an event and select a category.
- View their own enrolments.
- Track their own race results and performance history.

Keep these two roles consistent across your ERD, endpoint plan, database design and future Parts 2 and 3.

3. WHAT YOUR REPOSITORY SHOULD LOOK LIKE

By the time I mark Part 1, I should be able to open your GitHub repository and immediately understand where everything is stored.

```
RaceDay Repository
|
|-- README.md
|
|-- docs
|   |-- RaceDay_ERD.pdf
|   |-- RaceDay_API_Endpoint_Plan.pdf
|   `-- RaceDay_Database.sql
|
`-- .github
    `-- workflows
```

└─ part1-ci.yml

Your exact filenames may differ, but they must be clear, meaningful and consistent. The /docs folder must contain the ERD, endpoint plan and SQL script.

4. SECTION A - ENTITY RELATIONSHIP DIAGRAM (ERD)

Marks: 25

This is one of the highest-weighted sections in Part 1. I expect a complete relational database design for RaceDay, not a rough sketch.

Your ERD must show:

- A minimum of six entities.
- Attributes for the entities.
- Primary keys.
- Foreign keys.
- Relationships between entities.
- Cardinality for every relationship.
- Correct use of one-to-many and many-to-many relationship structures.

Step 1 - Identify the information RaceDay needs

Start with the functional requirements and ask what information the system must store. Do not begin by randomly inventing tables.

- What information must be stored about users?
- How will the system distinguish Organisers from Participants?
- What information must be stored for an event?
- How will event categories be represented?
- How will the database record which Participant entered which Event?
- How will the system store finish time and finishing position?
- How will an Event be linked back to the Organiser who created it?

A possible starting point could include entities such as User, Event, Category, Enrolment and Result. The assessment requires a minimum of six entities, so you must analyse the system further and decide what additional entity or entities are appropriate.

Do not add an entity simply to reach six. Every entity must have a clear purpose in RaceDay.

Step 2 - Decide on the attributes

For every entity, decide what information must be stored. For example, an Event may need the following information:

EventID
EventName
Description
EventDate
Location
Distance
EventType
OrganiserID

These are examples of the kinds of attributes you should consider. Your final design must be consistent with the requirements and your own ERD structure.

Step 3 - Identify the primary keys

Every table needs a value that uniquely identifies each record. Clearly mark primary keys on the ERD.

PK UserID
PK EventID
PK CategoryID
PK EnrolmentID
PK ResultID

Do not leave the marker guessing which attribute is the primary key.

Step 4 - Identify the foreign keys

Foreign keys connect related tables. For example, if an Event is created by an Organiser, the Event table may need an OrganiserID foreign key. If Enrolment connects a Participant to an Event and a Category, the Enrolment entity may need foreign keys to all three.

FK ParticipantID

FK EventID

FK CategoryID

Every foreign key should have a reason. Be prepared to explain the relationship in your video.

Step 5 - Show cardinality correctly

Clearly indicate relationship cardinality. Typical relationship types are:

1 : 1 One-to-one

1 : Many One-to-many

Many : Many

Examples that may make sense conceptually include:

- One Organiser can create many Events.
- One Event can have many Categories.
- One Participant can have many Enrolments.
- One Event can have many Enrolments.

If Participants can enter many Events and Events can have many Participants, this is a many-to-many relationship. In a relational database, you would normally resolve this through an associative entity such as Enrolment.

Step 6 - Check the ERD against the future application

Ask yourself: "Can the database represented by my ERD support everything RaceDay must eventually do?"

If the answer is no, fix the design before moving on. This is exactly why you should read all three PoE parts before starting Part 1.

Step 7 - Export and save the ERD

Export the completed ERD as a PNG or PDF and store it inside /docs. A clear filename would be:

docs/RaceDay_ERD.pdf

To reach the highest rubric band, the ERD should be complete and accurate, show all entities and attributes, identify primary and foreign keys, use correct cardinality and relationship types, and match the SQL script exactly.

5. SECTION B - API ENDPOINT PLAN

Marks: 25

In Part 1 you are planning the API. You are not coding controllers or building the Web API yet. Your job is to decide what endpoints the RaceDay API will need when you develop Part 2.

Your endpoint plan must cover at least:

- Authentication - registration and login.
- User Profile.
- Events.
- Categories.
- Event Enrolments.
- Results.

Step 1 - Create the endpoint plan table

Every endpoint row must contain all six required columns:

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/events	Creates a new RaceDay event.	Organiser	{ name, description, date, location,	201 Created; 400 Bad Request;

				distance, eventType }	401/403 if access is invalid
--	--	--	--	-----------------------	------------------------------

The example above illustrates the level of detail expected. Your actual plan should contain all the endpoints required by RaceDay.

Step 2 - Understand the HTTP Method column

GET = retrieve/read information
 POST = create something
 PUT = update something
 DELETE = remove something

Choose the method that correctly matches the action. Do not use POST for everything.

Step 3 - Write clear routes

Routes should be specific, consistent and start with /api/. Examples include:

/api/events
 /api/events/{id}
 /api/categories
 /api/enrolments

Keep your naming consistent across the full plan.

Step 4 - Write meaningful descriptions

Do not write a vague description such as "Gets event". Write a sentence that clearly explains the endpoint purpose. Example:

Returns the details of a specific RaceDay event using the supplied event ID.

Step 5 - Assign the correct role

Decide who should be allowed to call each endpoint. Typical values are:

None = Public
 Any = Any authenticated user
 Organiser = Organiser only
 Participant = Participant only

Think carefully about permissions. Both roles may be allowed to view events, but only Organisers should be able to create, update or delete events.

Step 6 - Define request bodies

For POST and PUT endpoints, identify the data that the client will send. For example, an event creation request may require:

```
{
  eventName,
  description,
  eventDate,
  location,
  distance,
  eventType
}
```

Most GET and DELETE requests will normally have no JSON request body.

Step 7 - Plan expected responses

Your endpoint plan should show both successful responses and realistic failure responses where relevant.

200 OK
 201 Created
 400 Bad Request
 401 Unauthorized
 403 Forbidden

404 Not Found

409 Conflict

Do not list every status code for every endpoint. Choose responses that make sense for that action.

6. MINIMUM API FUNCTIONALITY TO PLAN

Authentication

- Register a user.
- Log a registered user in.
- Consider how role information will be handled.

User Profile

- View own profile.
- Update own profile.

Events

- View all events.
- View one event.
- Create an event - Organiser.
- Update an event - Organiser.
- Delete an event - Organiser.

Categories

- View categories.
- Create categories for an event - Organiser.
- Plan update/delete category operations where your design requires them.

Event Enrolments

- Allow a Participant to enrol for an Event.
- Allow a Participant to choose a Category.
- Record the relationship between Participant, Event and Category.
- Allow an Organiser to view enrolments for their Events.

Results

- Allow an Organiser to capture finish time.
- Allow an Organiser to capture finishing position.
- Allow Participants to view their own results.

Once your endpoint plan is complete, save it as PDF or Markdown in /docs. A clear filename would be:

docs/RaceDay_API_Endpoint_Plan.pdf

For the highest rubric band, make sure every required resource is covered, every row uses all six columns, request bodies are clear, and success and failure response codes are included where appropriate.

7. SECTION C - SQL DATABASE SCRIPT

Marks: 20

You must convert your ERD into a working SQL Server database script using SQL Server Management Studio (SSMS).

Your SQL script must include:

- CREATE TABLE statements for every entity in your ERD.
- Primary keys.
- Foreign keys.
- NOT NULL constraints where appropriate.
- UNIQUE constraints where appropriate.
- DEFAULT constraints where appropriate.
- INSERT statements with realistic sample data.

Step 1 - Create every table represented by the ERD

If your ERD contains six or more entities, your SQL script must create corresponding tables for those entities. Do not design an entity in the ERD and then forget to create its table in SQL.

Step 2 - Define keys and constraints

A simple primary-key pattern may look like:

```
UserID INT IDENTITY(1,1) PRIMARY KEY
```

Required fields may use NOT NULL. Values such as email addresses may need UNIQUE where this suits your design. Relationships must be implemented with foreign keys.

Step 3 - Add realistic seed data

At minimum, your database script must seed the database with:

- 2 Organisers.
- 2 Participants.
- 3 Events.
- Categories for each Event.
- Sample Enrolments.

Use realistic RaceDay sample data rather than meaningless values such as Test1, ABC or XYZ. Your sample data should make the relationships easy to understand and test.

Step 4 - Test the full SQL script

The script must run without errors on a clean SQL Server instance. I recommend the following test process:

1. Open SSMS.
2. Start from a clean database environment.
3. Open your .sql file.
4. Execute the entire script from top to bottom.
5. Confirm that all tables are created.
6. Confirm that all primary and foreign keys are created.
7. Confirm that constraints are applied.
8. Confirm that all INSERT statements execute successfully.
9. Open the tables and verify the sample records.
10. Correct every error.
11. Run the complete script again to confirm it works cleanly.

Do not submit a script that only works because you manually created some tables beforehand. The .sql file itself should recreate the required structure.

Save the final script inside /docs. A clear filename would be:

```
docs/RaceDay_Database.sql
```

For the highest rubric band, the script should run cleanly on a fresh instance, match the ERD, define all required constraints and contain realistic seed data covering the system entities.

8. GITHUB - STEP-BY-STEP

Marks: 15

GitHub is part of the assessment. You are not submitting a ZIP file. You will submit the GitHub repository link on ARC, and your source/planning files must already be committed and pushed to the repository.

Step 1 - Accept the GitHub repository link

1. Open the GitHub link provided by the lecturer.
2. Sign in using your own GitHub account.
3. Accept or access the repository as instructed.
4. Open the repository and confirm it belongs to the correct assessment.

Use your own account because your commit history is used as evidence of your contribution.

Step 2 - Clone the repository to your computer

If you are not comfortable with command-line Git, I recommend GitHub Desktop.

1. Open the repository on GitHub.
2. Click Code.
3. Choose Open with GitHub Desktop, or copy the HTTPS link and clone it in GitHub Desktop.
4. Choose a local folder.
5. Click Clone.
6. Open the cloned folder on your computer.

You now have a local working copy of the same repository that appears online.

Step 3 - Create the required folders

Create the /docs folder and place the Part 1 planning files inside it. Also create the GitHub Actions workflow folder.

```
docs/
.github/workflows/
```

Step 4 - Commit continuously while you work

Do not finish the entire assessment and then upload everything with one final commit. Part 1 requires a minimum of 20 meaningful commits.

A meaningful commit represents a real unit of work. Examples include:

- Initialise RaceDay repository structure
- Add initial RaceDay ERD entities
- Add primary keys to ERD
- Add foreign key relationships to ERD
- Add cardinality to RaceDay ERD
- Refine enrolment relationship in ERD
- Add authentication endpoints to API plan
- Add event management endpoints
- Add category endpoints to API plan
- Add enrolment endpoints to API plan
- Add results endpoints to API plan
- Add API response status codes
- Add request bodies to endpoint plan
- Create Users table SQL script
- Create Events and Categories tables
- Create Enrolments table and foreign keys
- Create Results table and constraints
- Add RaceDay sample organisers and participants
- Add event and category seed data
- Add enrolment sample data
- Add Part 1 GitHub Actions workflow
- Add RaceDay project description to README
- Add Organiser and Participant role descriptions
- Add successful CI screenshot to README
- Add Part 1 video link to README

Avoid fake or weak commits such as "update file", "fixed typo", "change" or repeated meaningless changes simply to increase the number of commits. The commit history should show genuine progress.

9. HOW TO COMMIT AND PUSH USING GITHUB DESKTOP

1. Save your latest work.
2. Open GitHub Desktop.
3. Review the files shown under Changes.
4. In the Summary box, type a meaningful commit message. Example: Add event and category entities to ERD.
5. Click Commit to main.

6. Click Push origin.

7. Open the repository in your browser and confirm the commit is visible online.

A local commit is not enough. If you do not push it, I cannot see it on GitHub. Get into the habit of committing and pushing as you complete each meaningful section of work.

10. GITHUB ACTIONS / CI-CD

Part 1 requires a GitHub Actions workflow. Because you are not building the API yet, the workflow can validate the repository structure and check that the required Part 1 files exist.

Create:

`.github/workflows/part1-ci.yml`

Example workflow:

name: Part 1 Documentation Check

on:

push:

pull_request:

jobs:

validate-part1:

runs-on: ubuntu-latest

steps:

- name: Checkout repository

uses: actions/checkout@v4

- name: Check docs folder

run: test -d docs

- name: Check ERD

run: test -f docs/RaceDay_ERD.pdf

- name: Check API Endpoint Plan

run: test -f docs/RaceDay_API_Endpoint_Plan.pdf

- name: Check SQL Script

run: test -f docs/RaceDay_Database.sql

- name: Check README

run: test -f README.md

Important: If your filenames are different, change the workflow so that the file checks match your actual filenames.

Step-by-step: check the GitHub Action

1. Commit and push the workflow file.

2. Open the repository on GitHub.

3. Click the Actions tab.

4. Open the latest workflow run.

5. Wait for the workflow to complete.

6. Confirm that it shows a green check mark.

7. If the workflow is red, open the failed step, read the error, correct the problem and push again.

8. When the workflow is green, take a screenshot for the README.

The successful green build screenshot must be included in the README.

11. README FILE

Your repository must contain README.md. The README should help another person understand the project and how to inspect or run the Part 1 work.

Recommended structure:

RaceDay Event Management System

Project Description

Briefly explain RaceDay and the problem it solves.

User Roles

Organiser

Explain Organiser functionality.

Participant

Explain Participant functionality.

Part 1

- ERD

- API Endpoint Plan

- SQL Database Script

Repository Structure

Explain the /docs folder.

Database Setup

Explain how to open and run the SQL script in SSMS.

CI/CD

Explain what the GitHub Actions workflow checks.

Insert the successful green-build screenshot.

Video Demonstration

YouTube Link: YOUR_UNLISTED_LINK

For the highest Correct Submission mark, keep the repository structure clear, make the README detailed, include setup notes, ensure all required documents are in /docs, and include the CI screenshot and video link.

12. VIDEO PRESENTATION

Marks: 10

Every PoE part requires a video presentation. For Part 1, I expect you to demonstrate and explain your planning decisions, not merely scroll through files silently.

Recommended video flow:

1. Introduce the RaceDay system.
2. Show your GitHub repository.
3. Show the /docs folder.
4. Open your ERD.
5. Explain the entities.
6. Explain important relationships.
7. Explain primary keys and foreign keys.
8. Explain cardinality.
9. Open the API Endpoint Plan.
10. Explain the main groups of endpoints.
11. Explain why different roles have access to different endpoints.

12. Open the SQL script in SSMS.
 13. Explain the table structure.
 14. Explain the foreign-key relationships and constraints.
 15. Run the SQL script live in SSMS.
 16. Show that the script executes successfully.
 17. Show some of the seeded sample data.
 18. Show the GitHub Actions green build.
 19. Conclude by explaining how Part 1 prepares the system for Part 2.
- Use your own voiceover. Upload the video to YouTube as Unlisted and place the link in the README. The strongest videos thoroughly explain the ERD decisions, endpoint plan choices and SQL script design, and show the SQL script being run live in SSMS.

13. PART 1 MARK BREAKDOWN

Criterion	Marks	What I am looking for
Correct Submission	5	Correct repository structure, /docs folder, README, CI screenshot and video link.
ERD	25	Complete database design with entities, attributes, PKs, FKs, relationships and cardinality.
API Endpoint Plan	25	Complete plan covering required functionality and all six endpoint columns.
SQL Database Script	20	Working SQL script matching the ERD, with keys, constraints and realistic seed data.
GitHub and CI/CD	15	20+ meaningful commits and successful GitHub Actions validation.
Video Presentation	10	Professional explanation of the ERD, endpoint plan and SQL, including a live SSMS demonstration.
TOTAL	100	

14. RECOMMENDED ORDER OF COMPLETION

1. Read the entire PoE, including Parts 2 and 3.
2. Accept and clone the GitHub repository before starting major work.
3. Create /docs, .github/workflows and README.md.
4. Analyse the RaceDay functional requirements.
5. Create the first version of the ERD.
6. Add attributes, primary keys and foreign keys.
7. Add and verify cardinality.
8. Develop the API Endpoint Plan.
9. Complete all six columns for every endpoint.
10. Develop the SQL script from the ERD.
11. Add primary keys, foreign keys and constraints.
12. Add the required realistic seed data.

13. Run the full SQL script in SSMS.
14. Compare the final SQL design to the ERD and correct any mismatch.
15. Configure the Part 1 GitHub Actions workflow.
16. Fix the workflow until it produces a green build.
17. Complete the README.
18. Check that you have at least 20 meaningful commits.
19. Record the Part 1 video.
20. Upload the video to YouTube as Unlisted and add the link to README.
21. Make a final push to GitHub.
22. Open GitHub in the browser and verify every required file is visible.
23. Submit the GitHub repository link on ARC - not a ZIP file.

15. FINAL SUBMISSION CHECKLIST

Before submitting, I expect you to be able to answer YES to every item below.

- ☐ I have read all three parts of the PoE.
- ☐ I understand the RaceDay scenario.
- ☐ I am using my own GitHub account.
- ☐ I used the repository provided for the assessment.
- ☐ My repository contains a /docs folder.
- ☐ My ERD is inside /docs.
- ☐ My ERD contains at least six entities.
- ☐ My ERD contains appropriate attributes.
- ☐ My ERD clearly identifies primary keys.
- ☐ My ERD clearly identifies foreign keys.
- ☐ My ERD shows relationship cardinality.
- ☐ My ERD matches my SQL design.
- ☐ My API Endpoint Plan is inside /docs.
- ☐ My endpoint plan contains all six required columns.
- ☐ I planned Authentication endpoints.
- ☐ I planned User Profile endpoints.
- ☐ I planned Event endpoints.
- ☐ I planned Category endpoints.
- ☐ I planned Event Enrolment endpoints.
- ☐ I planned Results endpoints.
- ☐ My SQL script is inside /docs.
- ☐ My SQL script contains every required table.
- ☐ My SQL script contains primary keys.
- ☐ My SQL script contains foreign keys.
- ☐ My SQL script contains appropriate constraints.
- ☐ I added at least two Organisers.
- ☐ I added at least two Participants.
- ☐ I added at least three Events.
- ☐ I added Categories for the Events.
- ☐ I added sample Enrolments.
- ☐ I tested the full SQL script in SSMS.
- ☐ My SQL script executes without errors.
- ☐ I have at least 20 meaningful GitHub commits.
- ☐ All my work has been pushed to GitHub.
- ☐ I created a GitHub Actions workflow.
- ☐ My GitHub Actions workflow has a green build.
- ☐ I added the green-build screenshot to README.
- ☐ My README explains the RaceDay system.
- ☐ My README explains both user roles.

- ☐ My README contains setup information.
- ☐ I recorded my Part 1 video.
- ☐ My video explains my ERD decisions.
- ☐ My video explains my endpoint plan.
- ☐ My video explains my SQL design.
- ☐ I run the SQL script live during the video.
- ☐ My video is uploaded to YouTube as Unlisted.
- ☐ My YouTube link is in README.
- ☐ I checked the repository online before submitting.
- ☐ I am submitting the GitHub repository link on ARC, not a ZIP file.

16. FINAL MESSAGE TO STUDENTS

For Part 1, I am not looking for an API or MVC application yet. I am looking for evidence that you have properly planned the system you will eventually build.

Your three core technical deliverables are:

- ERD - tells me what data exists and how it is related.
- API Endpoint Plan - tells me how the future API will interact with that data.
- SQL Database Script - proves that your database design can actually be implemented.

These three deliverables should support one another. The ERD and SQL script should match, and the endpoint plan should make sense for the data and functionality you have designed.

GitHub then gives me evidence of how your work developed over time, while GitHub Actions verifies that the required repository structure is present. The video gives you an opportunity to explain your own reasoning and demonstrate that the SQL design works.

Do not rush to Part 2. A strong Part 1 will make Part 2 and Part 3 significantly easier because the database and API structure will already have been thought through carefully.

Source basis: PROG6212_POE (1).pdf - Part 1 System Planning and Database requirements and Part 1 marking rubric.